Atlas

Parametric flood protection for Houston

Fast cash in the first 72 hours, triggered by public gauge data, settled on Solana, priced on Tiger Data

---

20-Hour Build Plan – Version 3

CodeRED: Orion | Tracks: Overall, Pitch, Environment, Solana, Tiger Data, Vultr, GoDaddy

Team: 2 builders, TypeScript and Python only (no Rust)

As of October 10, 2026

The one-sentence pitch

Maria rents in Meyerland. When her bayou crosses flood stage, $500 lands on her phone before the water goes down. That pays for a hotel, food and a missed shift in the first 72 hours, when nothing else pays.

The bayou is named on stage only after Hour 1.5 confirms it triggers in the replay window (F22). Brays Bayou is the preferred choice because Meyerland sits on it.

Assumptions: a team of two, a 20-hour build inside a 24-hour clock that started at 1:00 pm CDT on October 10, devnet only, no custom Solana program. Items marked **[VERIFY]** must be checked by Gate G1. Tags such as [F7] point to the fix list in "Changes from v2".

Contents

Changes from v2 **2**

1 Product summary and scope **3**

2 System architecture **5**

3 Pre-flight checklist (first 30 minutes) **6**

4 Team roles and the API contract **6**

5 Hour-by-hour timeline **8**

6 Data layer: Tiger Data **9**

7 Pricing and capital model **13**

8 Solana layer (Person B) **15**

9 Backend, frontend and hosting **18**

10 Demo script **19**

11 Judge Q&A preparation **20**

12 Risk register, fallbacks and cut order **21**

13 Limitations of the product **22**

14 Submission checklist **23**

15 Unverified assumptions to check by Gate G1 **23**

Sources **24**

Changes from v2

v3 applies the 26 findings from the v2 audit plus four small consistency fixes. The money loop now survives repeated runs, proof moments are shown where they actually render, payout fairness holds across a whole storm, and the public site is locked down. Everything not listed is unchanged from v2.

#	v2 problem	v3 fix	Sec.
Critical			
F1	Reset left paid aUSD with holders; float covered one run; NAV fell cumulatively	Reset refills vault and float to a snapshot by minting aUSD; all pool numbers scoped to a run	6.6, 8.9
F2	Pool numbers read the vault while payouts came from the float	Ledger table; pool assets = vault + float; reconciled against chain	6.2, 7.4
F3	Server-signs-first purchase may break in Phantom	Signing order tested in Hour 0; Flow B (wallet signs first, server co-signs and submits) as fallback	8.3
F4	"PAID" and "+$500" assumed visible in Phantom	Phone view web app is the money moment; status written into the policy name; aUSD gets a name and symbol	8.1, 8.2, 9.2
F5	Live NWS alerts could freeze sales on stage; replay had no freeze	Demo mode reads replay-synthesized alerts keyed by run	7.7
F6	Public Start, Reset and Buy with no auth	Admin token, rate limits, demo wallet allowlist, per-wallet reservation cap	9.3
High			
F7	Pay factor fair within a tick-defined wave, unfair across waves	Storm-level interim payment at a conservative factor, then a true-up when the storm closes	7.5
F8	Trigger re-fired after 72 h while still above threshold; pricing used a different event rule	One event definition everywhere: open until 72 h continuously below trigger	6.4, 7.1
F9	Time shift created future-dated rows; `now()` comparisons broke	Separate `event_time` and `wall_time`; a run clock replaces `now()`	6.2, 6.5
F10	Capital charge k undefined; only one tail scenario	Historical-year scenario set across all gauges gives a correlated loss distribution	7.3
F11	Stage history may not be comparable across datum or channel changes	Datum and channel-change check per gauge in Hour 0; truncate or adjust the record	6.1, 7.1
F12	Crash between send and record could double pay	Sign, write intent with signature, then send; restart checks signature status	8.6
F13	Longer status values need extra rent	Fixed-width status codes and pre-funded metadata space	8.2
F14	Switchboard would read a moving replay cursor	Feed endpoint returns the latched triggering value per event	8.10
F15	Evidence hash format undefined	Canonical preimage string specified; preimage stored next to hash	8.7

#	v2 problem	v3 fix	Sec.
F16	Person B overloaded; Python building Token-2022 transactions	One TypeScript chain service owns all signing (B); FastAPI read-only plus hosting (A)	4, 9
Medium			
F17	G2 needed the full streamer and reset	G2 proven with a minimal insert script	5
F18	New domain DNS can lag	Register in the first 30 minutes; `sslip.io` fallback hostname	3, 9.3
F19	Old compression API names	Columnstore API with fallback to the old one	6.3
F20	Replay reading count did not match the window	Count read from data; speed derived from it	6.5
F21	Buying on a phone can hang on stage	One pre-bought backup policy per demo	10
F22	Pitch named Brays before checking it triggers	Gauge named after Hour 1.5	1.1, 10
F23	Late start not handled	Late-start rule and compressed timeline	5.3
F24	Seven tracks can eat build time	Read Vultr and GoDaddy rules first; sidetrack work capped	1.5
F25	"How does Maria cash out?" unanswered	Honest answer prepared; listed as a limitation	11, 13
F26	Seeded reserve looked like premium income	Labelled "seeded capital" on the pool page	7.6
Consistency			
C1	NAV added coupons that the waterfall said were paid out	Coupons accrue into NAV; no distributions in the demo	7.4
C2	"Waves paid in order" limitation	Rewritten for interim and true-up	13
C3	Unclear which frequency estimate sets the price	Rule stated: higher of annual-peak and declustered rates	7.1
C4	`wave_id` tied to engine ticks	Replaced by `storm_id`	6.2

1 Product summary and scope

Atlas pays a fixed sum to every covered household near a Houston bayou gauge within minutes of that gauge crossing flood stage. No claim form, no adjuster.

1.1 The pitch, led by a person

Maria rents in Meyerland. When her bayou crosses flood stage, $500 lands on her phone before the water goes down. That pays for a hotel, food and a missed shift in the first 72 hours, when nothing else pays. Keep the technology out of the first 30 seconds. Name the bayou only after it is confirmed as a demo gauge **[F22]**.

1.2 The problem

US flood insurance runs mostly through the federal program, which has a waiting period before cover starts and claims that take weeks or months. Many Houston renters and homeowners have no flood cover at all. In the first 72 hours people need cash for a hotel, a car, food and lost wages.

1.3 What Atlas does

Atlas sells a small policy tied to the nearest stream gauge. If that gauge crosses a set flood level during the term, the policyholder gets a fixed payout (for example $500). In a storm big enough to strain the pool, part is paid immediately and the rest when the storm closes [F7]. Atlas does not pay for property damage and does not replace flood insurance.

1.4 How each sponsor technology is used

Tiger Data	Gauge history and live readings as hypertables; continuous aggregates; column-store compression; event state machine; pricing from correlated historical-year scenarios; the money ledger; replay engine; simulated 2,000-policy book.
Solana	aUSD test stablecoin; policies as non-transferable Token-2022 tokens with on-chain metadata; atomic purchase; Squads multisig vault; payout bot with intent records; Memo evidence with a canonical hash; catastrophe bond token. Optional: Switchboard feed, spending limit, Blink.
Vultr	Hosts API, chain service, bot, replay feed and dashboard over HTTPS. Optional: Object Storage for the raw data cache and policy terms JSON, if the track rules reward it **[VERIFY]**.
GoDaddy Registry	Domain for the hosted site.

1.5 Tracks entered

Best Overall, Best Pitch/Demo, Best Environment Hack, Best Use of Solana, Best Use of Tiger Data, Best Use of Vultr, Best Domain Name from GoDaddy Registry.

Side-track rule [F24]

Read the Vultr and GoDaddy Registry rules in the first 30 minutes. If GoDaddy requires a specific promotion or top-level domain, use it. Side-track work is capped at 1 hour total beyond hosting the site and pointing the domain. Confirm on Devpost that one project can enter all seven tracks.

1.6 In scope for the 20 hours

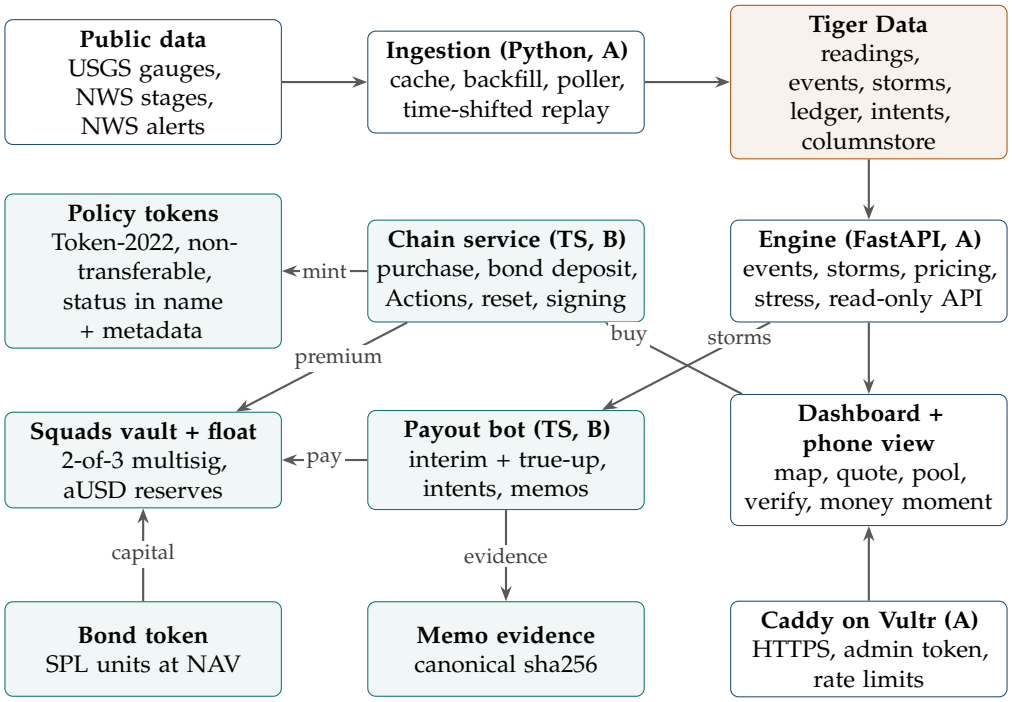
6 to 10 gauges with cached history and a live poller; pricing with confidence bands and a correlated capital charge; atomic purchase; storm-level payouts with interim and true-up; ledger-based pool accounting; bond token with NAV; a time-shifted Harvey replay that can run back to back; hosted, authenticated dashboard with a phone view.

1.7 Out of scope

Mainnet, real money, a custom Solana program, KYC, a native mobile app, rainfall triggers, property-level flood modelling, cash-out to bank accounts, and any claim to be a licensed insurance product.

2 System architecture

Every money movement is a row in the ledger and a transaction on chain, and the two reconcile after every run [F2].



2.1 Data flow for one storm

1. The poller or replay streamer writes readings with `event_time`, `wall_time`, `source` and `run_id` [F9].
2. The engine opens an *event* for a gauge after two consecutive accepted readings at or above `trigger_ft`, if that gauge has no open event in this run [F8].
3. The first event in a run opens a *storm*. Later events join the open storm [F7].
4. The engine latches the triggering value and computes the canonical evidence hash [F14][F15].
5. The bot pays every eligible policy on that gauge the *interim* amount $f_{\text{int}} \cdot B$ (7.5), recording an intent with the signature before sending [F12].
6. Each payout writes a Memo, sets the policy status to `PAID`, and writes ledger rows.
7. An event closes after 72 hours of event time continuously below trigger. The storm closes when all its events are closed, or when the replay ends.
8. On storm close, the bot pays the *true-up* $(f_{\text{fin}} - f_{\text{int}}) \cdot B$ to every paid policy, if positive.

9. After the run, the reconciliation check confirms ledger totals match on-chain balances **[F2]**.

3 Pre-flight checklist (first 30 minutes)

Do these first, in this order (A):

- ☐ Register the GoDaddy Registry domain and point DNS at the Vultr IP now, so propagation runs while you build. Note the `sslip.io` fallback hostname for the IP **[F18]**.
- ☐ Read the Vultr, GoDaddy and Devpost multi-track rules **[F24]**.
- ☐ Vultr instance up; free credits claimed; Caddy installed.
- ☐ Tiger Cloud service running; connection tested from both laptops; note free-tier limits and the TimescaleDB version (needed for F19). Local fallback: `timescale/timescaledb` Docker image.
- ☐ Generate `ADMIN_TOKEN`; store it outside the repo **[F6]**.

Chain setup (B):

- ☐ Devnet RPC provider key. Devnet SOL in every wallet: 3 policyholders, 1 investor, bot, 3 multisig members, mint authority.
- ☐ aUSD: classic SPL mint, 6 decimals; 1,000,000 minted to treasury. Add a name and symbol ("Atlas Test USD", "aUSD") via token metadata so wallets show something readable **[VERIFY]** **[F4]**.
- ☐ Phone: Phantom in Testnet Mode, funded, screen lock off. Open the phone view URL and add it to the home screen.
- ☐ Pin `@solana/web3.js@1.x`; do not install `@solana/kit`.

Code setup: monorepo `/data`, `/engine`, `/chain`, `/web`, `/docs`; `.env.example`; Node 20+, Python 3.11, Solana CLI on both laptops. API contract written to `/docs/api.md`; canonical evidence format written to `/docs/evidence.md`.

4 Team roles and the API contract

Person A owns every number, the database and the server. Person B owns every transaction and every pixel **[F16]**.

4.1 Roles

Person A: data, risk, hosting (Zeke)	Tiger schema, ingestion, event and storm logic, replay and run clock, pricing, scenario set, simulated book, ledger and reconciliation, FastAPI (read-only), NWS freeze, Vultr, Caddy, domain, deploys, auth and rate limits.
Person B: chain and product	Chain service in TypeScript (purchase, bond deposit, Actions, chain side of reset), all signing, policy and aUSD mints, Squads vault, payout bot, Memo evidence, dashboard, phone view, upgrades.
Shared	Gates, pitch, Q&A, fallback video, Devpost write-up.

Language boundary

Python never builds or signs a Solana transaction. The chain service is the only process holding signing keys apart from the bot. FastAPI reads Tiger and serves JSON; the chain service writes ledger and policy rows through a small set of SQL functions both sides agree on in Hour 1.

4.2 API contract (`/docs/api.md`, Hour 1)

B builds the UI against mock JSON with these shapes until A's endpoints are live.

Endpoint	Returns	Owner
`GET /gauges?run_id=`	`[{gauge_id, name, lat, lon, trigger_ft, latest_ft, status, demo_ready}]`; status normal / near / triggered	A (FastAPI)
`GET /gauges/{id}/series?run_id=`	`[{event_time, stage_ft}],trigger_ft, rejected_count`	A
`GET /quote?gauge=&payout=&run_id=`	`{premium, p_payout, band:[p5,p95], k, waiting_days, sales_open, reason}`	A
`GET /pool?run_id=`	`{onchain:{seeded_capital, premiums_in, reserve, bond_capital, float, nav_per_unit, payouts_out, reconciled}, simulated:{...}}`	A
`GET /storms?run_id=`	`[{storm_id, status, f_int, f_fin, events:[{event_id, gauge_id, opened_at, closed_at, value, evidence_hash, preimage}], payouts:[{policy, kind, amount, tx_sig}]}]`	A reads, B writes payouts
`GET /stress?scenario=`	`harvey` or `worst_year`: `{aggregate_loss, reserve_used, bond_used, f_fin, investor_loss_pct}`	A
`GET /ledger?run_id=`	ledger rows and totals by account	A
`POST /replay/start`, `POST /replay/reset`	`{run_id}`; requires `X-Admin-Token`	A, calls B's reset
`GET /policies?owner=`	`[{policy_mint, gauge_id, payout, status, paid_amount}]`	B (chain service)
`GET/POST /api/actions/buy?gauge=`	Action metadata; then a transaction (Flow A or B, 8.3)	B
`POST /api/purchase/submit`	Flow B only: wallet-signed tx in, co-signed and sent	B
`POST /api/bond/deposit`	Bond deposit transaction	B
`POST /chain/reset`	Internal only; refills balances, resets statuses	B

5 Hour-by-hour timeline

Hour 0 is 1:00 pm CDT, October 10. The Devpost deadline is Hour 24 (1:00 pm CDT, October 11). The plan submits at Hour 19.5.

5.1 Phases and gates

Gates

G1 (Hour 1.5): every **[VERIFY]** item green or its fallback written in `/docs/decisions.md`, including the Phantom signing order and which bayous trigger in the replay window. **G2 (Hour 5):** a minimal insert script [F17] pushes a gauge over trigger in a test run; a real devnet transaction pays a real policy; ledger and chain agree. **G3 (Hour 16, freeze):** on the hosted URL, start, run, reset, run again, twice in a row, with no code changes; reconciliation passes after each run.

5.2 Timeline

Hours	Person A (data, risk, hosting)	Person B (chain and product)
0 to 0.5	Domain and DNS; track rules; Vultr and Caddy; Tiger Cloud; admin token	RPC key; devnet SOL; aUSD mint and metadata; phone Phantom set up
0.5 to 1.5	USGS legacy and new API checks; choose gauges with real site numbers; datum and channel-change check [F11]; cache raw JSON; create schema; identify which gauges cross trigger in the Harvey window	2-of-3 Squads multisig and vault aUSD account; Token-2022 policy test (extensions, name update, fixed-width status, rent); Phantom signing-order test, Flow A vs Flow B [F3]; mock JSON
G1 (Hour 1.5)		
1.5 to 4	Backfill Harvey window and annual peaks from cache; live poller; continuous aggregates; columnstore policy; ledger, intents, storms tables; HTTPS live on domain or `sslip.io`	Chain service skeleton (TS); pre-created policy mints; purchase in the chosen flow, from a script; ledger rows on purchase
4 to 5	Event open logic; run clock; minimal insert script for G2	Simple bot: intent, sign, send, Memo, status `PAID`, ledger
G2 (Hour 5): if it fails, cut scope now.		
5 to 8	Full replay streamer with time shift; event close; storm logic; pricing, bootstrap band; historical-year scenarios and k; simulated book	Chain side of reset (refill, status reset, release mints); dashboard and phone view against mocks
8 to 10	Interim and true-up factors; stress tests; bond sizing; NWS freeze, live and replay; deploy FastAPI; admin token and rate limits in Caddy and FastAPI	Bond deposit; true-up payouts in bot; swap mocks for the real API; deploy chain service, bot and web

Hours	Person A (data, risk, hosting)	Person B (chain and product)
10 to 11	Integration and food. Run start, reset, start on the hosted URL once. Tag `v0.1`.	
11 to 14	Replay polish; reconciliation check script; `/docs/numbers.sql`	Upgrades in 1-hour boxes: QR or Blink; spending limit; Switchboard (latched feed). Then verify page
14 to 16	Staggered rest: A rests 14 to 15 while B polishes the phone view; B rests 15 to 16 while A runs the numbers audit.	
G3 (Hour 16, feature freeze)		
16 to 18	Slides with pricing, scenario and stress numbers; Q&A drill	Two rehearsals each of the 90-second and 4-minute scripts; record fallback video
18 to 19.5	README, diagram, setup and reset steps	Devpost page, screenshots, tracks, address list
19.5 to 24	Submitted. Sleep in shifts; reset and one full run 20 minutes before judging.	

Rule for the whole build

Blocked for 45 minutes: say so out loud and switch to the fallback (section 12). An upgrade that misses its 1-hour box is dropped, not extended.

5.3 Late-start rule

If you begin this plan after Hour 0, the deadline does not move. Let h be the hour you start.

- $h \leq 1.5$: start at the row for the current hour; do the pre-flight items you skipped in parallel.
- $1.5 < h \leq 4.5$: take the lost hours out of the upgrade block (Hours 11 to 14) first, then the rest block. Gates move by the same amount; submission stays at Hour 19.5.
- $h > 4.5$: drop all three upgrades and the verify page; keep the never-cut list (section 12). Submission moves no later than Hour 21.

6 Data layer: Tiger Data

6.1 Sources and checks

- **USGS legacy WaterServices:** instantaneous stage and discharge, about every 15 minutes, plus annual peaks. Being retired (see Sources), so cache everything in Hour 1 and never call it on stage.
- **USGS new API:** check in Hour 0 which endpoints cover your gauges; use it for the live poller if the legacy service is degraded.
- **NWS flood stages:** set `trigger_ft` **[VERIFY]**.
- **NWS alerts:** live sales freeze (7.7) **[VERIFY]**.
- **Cache rule:** every raw response written to `/data/cache/` before parsing.

Gauge selection: 6 to 10 gauges on major bayous with live data, at least 20 years of peaks and data through Harvey. Read from the stored data which ones cross `trigger_ft` in the replay window; set `demo_ready = true` on four of them **[F22]**.

Comparability check [F11]: for each gauge, read the site's datum history and note any channel project on that bayou. If the datum changed, convert or truncate to the consistent period. If a large channel project changed the stage-flood relationship, price from the post-project period or from discharge, and record the decision per gauge in `gauges.record_note`.

6.2 Schema

```
-- Readings: event_time is the (possibly shifted) hydrologic time;
-- wall_time is when the row was written. Replay never compares to now().
CREATE TABLE gauge_readings (
  event_time TIMESTAMPTZ NOT NULL,
  wall_time TIMESTAMPTZ NOT NULL DEFAULT now(),
  gauge_id TEXT NOT NULL,
  stage_ft DOUBLE PRECISION,
  discharge_cfs DOUBLE PRECISION,
  source TEXT NOT NULL, -- 'usgs' | 'replay'
  run_id TEXT NOT NULL DEFAULT 'live',
  rejected BOOLEAN NOT NULL DEFAULT false
);
SELECT create_hypertable('gauge_readings', by_range('event_time'));
CREATE UNIQUE INDEX ON gauge_readings (gauge_id, event_time, source, run_id);

CREATE TABLE gauges (
  gauge_id TEXT PRIMARY KEY, name TEXT, lat DOUBLE PRECISION, lon DOUBLE PRECISION,
  trigger_ft DOUBLE PRECISION, nws_flood_ft DOUBLE PRECISION,
  record_start INT, record_note TEXT, -- F11 comparability decision
  feed_pubkey TEXT, demo_ready BOOLEAN DEFAULT false
);

CREATE TABLE annual_peaks (
  gauge_id TEXT, water_year INT, peak_stage_ft DOUBLE PRECISION,
  peak_time TIMESTAMPTZ, PRIMARY KEY (gauge_id, water_year)
);

CREATE TABLE runs (
  run_id TEXT PRIMARY KEY, source TEXT NOT NULL,
  started_wall TIMESTAMPTZ NOT NULL, shift INTERVAL,
  clock TIMESTAMPTZ, -- run clock (F9)
  status TEXT NOT NULL -- running | done | reset
);

CREATE TABLE storms (
  storm_id TEXT PRIMARY KEY, run_id TEXT NOT NULL,
  opened_at TIMESTAMPTZ NOT NULL, closed_at TIMESTAMPTZ,
  f_int NUMERIC, f_fin NUMERIC, status TEXT NOT NULL -- open | closed | trued_up
);

CREATE TABLE gauge_events (
  event_id TEXT PRIMARY KEY, run_id TEXT NOT NULL, storm_id TEXT NOT NULL,
  gauge_id TEXT NOT NULL, opened_at TIMESTAMPTZ NOT NULL, closed_at TIMESTAMPTZ,
  latched_value DOUBLE PRECISION NOT NULL, -- F14
  feed_pubkey TEXT, feed_slot BIGINT,
  preimage TEXT NOT NULL, evidence_hash TEXT NOT NULL -- F15
);
CREATE UNIQUE INDEX one_open_event ON gauge_events (run_id, gauge_id)
```

```
  WHERE closed_at IS NULL; -- F8: one open event per gauge

CREATE TABLE policies (
  policy_mint TEXT PRIMARY KEY, owner TEXT NOT NULL,
  gauge_id TEXT REFERENCES gauges, payout_usd NUMERIC NOT NULL,
  premium_usd NUMERIC NOT NULL, starts_at TIMESTAMPTZ NOT NULL,
  ends_at TIMESTAMPTZ NOT NULL, status CHAR(4) NOT NULL, -- ACTV | PAID | EXPD
  paid_amount NUMERIC NOT NULL DEFAULT 0, demo BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE sim_policies (
  sim_id BIGINT PRIMARY KEY, gauge_id TEXT REFERENCES gauges,
  payout_usd NUMERIC, premium_usd NUMERIC
);

-- F2: every money movement, double entry
CREATE TABLE ledger (
  wall_time TIMESTAMPTZ NOT NULL DEFAULT now(), run_id TEXT NOT NULL,
  entry_id TEXT NOT NULL, account TEXT NOT NULL,
  -- seeded | premiums | reserve | bond | float | holders | expenses | coupon
  amount NUMERIC NOT NULL, -- signed; each entry sums to 0
  memo TEXT, tx_sig TEXT
);
SELECT create_hypertable('ledger', by_range('wall_time'));

-- F12: written before any payout is sent
CREATE TABLE payout_intents (
  intent_id TEXT PRIMARY KEY, -- hash(policy_mint, storm_id, kind)
  run_id TEXT NOT NULL, storm_id TEXT NOT NULL, policy_mint TEXT NOT NULL,
  kind TEXT NOT NULL, -- interim | trueup
  amount NUMERIC NOT NULL, tx_sig TEXT NOT NULL,
  status TEXT NOT NULL, -- signed | sent | confirmed | expired
  created_wall TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE replay_alerts ( -- F5
  run_id TEXT, gauge_id TEXT, starts_at TIMESTAMPTZ, ends_at TIMESTAMPTZ, kind TEXT
);
```

6.3 Continuous aggregates and columnstore

```
CREATE MATERIALIZED VIEW gauge_hourly
WITH (timescaledb.continuous, timescaledb.materialized_only = false) AS
SELECT time_bucket('1 hour', event_time) AS bucket, gauge_id, source, run_id,
     max(stage_ft) AS max_stage, avg(stage_ft) AS avg_stage,
     max(discharge_cfs) AS max_q, count(*) AS n
FROM gauge_readings WHERE NOT rejected
GROUP BY bucket, gauge_id, source, run_id;

SELECT add_continuous_aggregate_policy('gauge_hourly',
  start_offset => INTERVAL '3 days', end_offset => INTERVAL '15 minutes',
  schedule_interval => INTERVAL '15 minutes');

-- Current API (TimescaleDB 2.18+), VERIFY on your Tiger service:
ALTER TABLE gauge_readings SET (
  timescaledb.enable_columnstore = true,
  timescaledb.segmentby = 'gauge_id, source',
  timescaledb.orderby = 'event_time DESC');
CALL add_columnstore_policy('gauge_readings', after => INTERVAL '7 days');
```

```
-- Stats for the slide: SELECT * FROM hypertable_columnstore_stats('gauge_readings');
-- Fallback if the service is older: timescaledb.compress / add_compression_policy.
```

History charts read `gauge_hourly`. The live replay chart reads `gauge_readings` for the current `run_id`. Replay rows are future-dated relative to wall time and stay in the rowstore; the decades of backfill are what compress.

6.4 Event logic

One rule for pricing and for the contract: an event *opens* after two consecutive accepted readings at or above `trigger_ft`; it *closes* once the gauge has stayed below `trigger_ft` for 72 hours of event time. A gauge can have at most one open event per run (enforced by the partial unique index).

Open:

```
WITH last2 AS (
  SELECT g.gauge_id, g.trigger_ft, r.event_time, r.stage_ft
  FROM gauges g
  CROSS JOIN LATERAL (
    SELECT event_time, stage_ft FROM gauge_readings
    WHERE gauge_id = g.gauge_id AND source = $1 AND run_id = $2 AND NOT rejected
    ORDER BY event_time DESC LIMIT 2) r
)
SELECT gauge_id, max(event_time) AS opened_at, max(stage_ft) AS value
FROM last2
GROUP BY gauge_id, trigger_ft
HAVING count(*) = 2 AND min(stage_ft) >= trigger_ft
   AND NOT EXISTS (SELECT 1 FROM gauge_events e
                WHERE e.run_id = $2 AND e.gauge_id = last2.gauge_id
                  AND e.closed_at IS NULL);
```

Close: for each open event, close it at the run clock when no accepted reading in the last 72 hours of event time is at or above trigger, and at least one reading exists past that window:

```
UPDATE gauge_events e SET closed_at = $3 -- $3 = run clock
WHERE e.run_id = $2 AND e.closed_at IS NULL
  AND NOT EXISTS (SELECT 1 FROM gauge_readings r
                JOIN gauges g USING (gauge_id)
                WHERE r.gauge_id = e.gauge_id AND r.run_id = $2 AND NOT r.rejected
                  AND r.event_time > $3 - INTERVAL '72 hours'
                  AND r.stage_ft >= g.trigger_ft);
```

End of replay: the replay window ends before most events would close, so the streamer's final step closes all open events and the storm at the run clock. The demo says "storm closed" aloud.

Data-quality guard: computed in Python before insert; a reading that jumps more than a set number of feet from the previous accepted reading within 15 minutes is stored with `rejected = true`. The gauge page shows the rejected count.

6.5 Run clock and replay

- **Run clock.** For a live run it is `now()`. For a replay it is the `event_time` of the last reading written. Every query that previously used `now()` (latest reading, policy active, sales freeze, event close, Switchboard freshness relative to the event) takes the run clock as a parameter. Grep the

codebase for `now()` and `Date.now()` at Hour 10; the only allowed uses are `wall_time` defaults and transaction timing.

- **Window.** From 48 hours before the first demo gauge crosses trigger to 24 hours after the last peak, read from the stored data. At 15-minute spacing a 5 to 6 day window is roughly 480 to 580 readings per gauge; use the actual count **[F20]**.
- **Speed.** `REPLAY_SECONDS = 90`; seconds per reading = `REPLAY_SECONDS / count`.
- **Shift.** `event_time = original_time + (started_wall − window_start)`. The first reading is stamped at the moment the run starts, so a policy bought a minute earlier is active. Later readings carry future event times; that is intended and why nothing compares event time to `now()`.
- **Policy activity** is judged against the run clock: active when `starts_at <= clock < ends_at`. Demo policies are bought with term covering at least 30 days so the future-dated replay stays inside it.

6.6 Reset

`POST /replay/reset` (admin token required) runs, in order:

1. Stop any running streamer; mark the run `reset`.
2. Chain service `POST /chain/reset`: read on-chain balances of vault and float; mint aUSD to each to restore the snapshot taken at setup (for example vault reserve 300, bond 5,000, float 2,000); set every demo policy's status and name back to `ACTV`; release mint reservations.
3. Write ledger entries for the refill under account `seeded`, tagged with the reset run.
4. Mark demo policies `ACTV`, `paid_amount = 0` in Tiger.
5. Run reconciliation; return `reconciled: true` or the mismatch.

Pool and NAV figures are always computed for one `run_id` from the ledger, starting from the snapshot, so a fresh run shows a fresh pool.

Reset rule

The holders keep the aUSD they received; reset creates new aUSD rather than taking it back. On a test mint this is harmless, and it must be said if anyone asks why total supply grows.

7 Pricing and capital model

7.1 Event frequency

For gauge g with T_g comparable years of record:

- $\hat{\lambda}_g^{\text{peak}} = N_g^{\text{peak}}/T_g$, where N_g^{peak} is the number of years whose annual peak is at or above `trigger_ft`. At most one event per year, so it can understate.
- $\hat{\lambda}_g^{\text{iv}}$, from instantaneous data for the years it exists, counting events with the same open/close rule as the contract.

Price with the higher of the two [FC3]. Say which one set each gauge's price on the slide.

7.2 Premium

Fixed benefit B, paid at most once per term t (interim plus true-up together never exceed B):

$$P(\text{payout}) = 1 - e^{-\hat{\lambda}_g t}, \qquad \mathbb{E}[L] = B\,(1 - e^{-\hat{\lambda}_g t}), \qquad \pi = \mathbb{E}[L](1+\theta) + c\,k.$$

Bootstrap the record 2,000 times and show π with a 5th to 95th percentile band.

7.3 Correlated capital and k

Each historical year y is one scenario, using that year's annual peaks at *all* gauges together, which keeps the cross-gauge correlation that a single gauge's history cannot show. For the simulated book:

$$S_y = \sum_{i \in \text{book}} B_i\, \mathbf{1}\{\text{peak}_{g(i),y} \geq \text{trigger}_{g(i)}\}.$$

Add the Harvey scenario if Harvey's year is outside the comparable record. Required capital is

$$K = \max\left(\max_y S_y,\ S_{\text{Harvey}}\right) - \overline{S},$$

the worst observed year above the mean annual loss. With 30 to 40 scenarios a 99.5% quantile is not estimable, so state that K is a worst-observed-year standard, not a modelled tail. Each policy's share is $k_i = K \cdot B_i / \sum_j B_j$.

7.4 Waterfall

Each premium π is split once: expenses $e\pi$; bond coupon $c\,k$, which **accrues into bond NAV** (no distributions in the demo); the remainder to reserve R. Losses hit R, then bond capital C, then the pay factor.

$$\text{NAV}_t = \frac{C_0 - \text{losses absorbed}_t + \text{coupons accrued}_t}{\text{units outstanding}}.$$

All terms come from ledger rows for the current run **[F2]**. Pool assets are vault plus float; the float is not a loss.

7.5 Storm payouts: interim and true-up

A storm groups every gauge event in a run from the first open until all are closed. Let A be available funds ($R + C$ at storm open), N_{risk} the number of active policies on all gauges in the storm's at-risk set (all Houston gauges in this build), and N_{trig} the policies whose gauges actually triggered by storm close.

$$f_{\text{int}} = \min\left(1,\ \frac{A}{B \cdot N_{\text{risk}}}\right), \qquad f_{\text{fin}} = \min\left(1,\ \frac{A}{B \cdot N_{\text{trig}}}\right) \geq f_{\text{int}}.$$

Every triggered policy receives $f_{\text{int}} B$ immediately, whatever order its gauge crossed in. At storm close each receives $(f_{\text{fin}} - f_{\text{int}})B$. Everyone in the storm ends with the same share. On the demo's on-chain pool A is large relative to three policies, so $f_{\text{int}} = 1$; the simulated book shows a real interim cut and true-up.

7.6 Two pool panels

Panel	Contents	Source
On-chain (devnet)	Seeded capital (labelled "seeded", not premiums), premiums from demo purchases, 3 demo policies, reserve, bond capital, float, NAV, reconciliation status	Ledger for the run, checked against vault, float and mint balances
Simulated book	2,000 policies; worst-year and Harvey losses; layer use; f_{int} and f_{fin}; investor loss	`sim_policies`, `/stress`

Sizing target for the on-chain panel: seeded reserve 300 aUSD, bond 5,000 aUSD; three $500 payouts use the reserve and the bond absorbs about 1,200 plus premiums net of expenses, so NAV visibly falls. Set final figures from A's pricing.

7.7 Underwriting rules

- Waiting period 7 days; demo mode sets 0 and the demo says so.
- **Sales freeze:** live mode reads NWS alerts for the gauge's point. Replay mode reads the `replay_alerts` table for the run, generated when the run starts (a flood watch from 24 hours of event time before the first crossing). The live API is never consulted in demo mode, so a real watch on judging day cannot block the purchase.
- One policy per wallet per gauge; a cap per wallet.

8 Solana layer (Person B)

All signing lives in the TypeScript chain service and the bot [F16]. The simple version (8.1 to 8.9) uses token programs, Memo and a plain Squads multisig, and must work by G2. Upgrades come after (8.10).

8.1 aUSD

Classic SPL Token mint, 6 decimals, 1,000,000 to treasury. Give it a name and symbol through token metadata so Phantom shows "aUSD" rather than an unknown address **[VERIFY]**. Policies use Token-2022 and aUSD uses the classic Token program: derive every associated token account with the right program ID.

8.2 Policy tokens

- Mint with supply 1, 0 decimals. Extensions: NonTransferable, MetadataPointer (to itself), TokenMetadata; PermanentDelegate only if the Hour-0 test passes (burn at expiry only).
- Initialization order as in v2: size the account for fixed extensions, fund lamports for metadata too, initialize extensions, `initializeMint`, then metadata.
- **Status is visible in the name:** `name = "Atlas Policy ACTV"` or `"Atlas Policy PAID"`. Phantom shows the name; it does not show custom fields **[VERIFY]**.

- **Fixed width:** status codes are always four characters (`ACTV`, `PAID`, `EXPD`) in both the name and the `status` field, so updates never grow the account. Pre-fund 256 extra bytes of rent anyway.
- Minimal fields: `policy_id`, `gauge_id`, `status`, `terms_hash`, `uri` to the terms JSON.
- Pre-created pool of about 10 unminted policy mints per demo gauge.
- Payout updates status to `PAID`; burn only at expiry; reset sets `ACTV`.

8.3 Atomic purchase

One transaction: create the buyer's Token-2022 account (idempotent); `transferChecked` premium in aUSD to the vault's token account; `mintTo` one policy token; remove mint authority. The backend reserves one pre-created mint for 2 minutes with a row lock.

Flow A (preferred if it passes the Hour-0 test)	Server partially signs as mint authority and returns the transaction; the wallet signs as fee payer and sends. Works for the Buy button, Solana Pay and Blinks.
Flow B (fallback)	Wallet calls `signTransaction` only; the client posts the signed transaction to `/api/purchase/submit`; the chain service adds the mint-authority signature and submits. Works for the Buy button and the phone view; Blinks are dropped.

Hour-0 test: run Flow A from the real phone in Phantom Testnet Mode. If Phantom rejects it, warns, or the transaction fails signature verification, switch to Flow B and record the decision in `/docs/decisions.md`. Sign promptly; a blockhash expires in about a minute.

8.4 Pool: Squads multisig

2-of-3 Squads v4 multisig; the vault is a PDA, so its aUSD token account uses `allowOwnerOffCurve = true`; time lock about 60 seconds in demo mode. One approved proposal funds a capped bot float (2,000 aUSD). The float is pool money held by the bot and counts in pool assets **[F2]**.

8.5 Payout bot

1. Poll for new events and storm closes in the current run.
2. For each eligible policy (on-chain check: holder holds it, status `ACTV` or already `PAID` for a true-up, gauge matches), compute the amount: interim $f_{\text{int}}B$, or true-up $(f_{\text{fin}} - f_{\text{int}})B$.
3. Build the transaction (create holder aUSD account idempotently, `transferChecked` from the float, Memo with the evidence), sign it, and go to 8.6 before sending.
4. On confirmation: set status `PAID` (interim only), add to `paid_amount`, write ledger rows (float $\rightarrow$ holders; reserve or bond absorbs).

8.6 Idempotency

1. `intent_id = sha256(policy_mint | storm_id | kind)`. If an intent with this id is `confirmed`, skip.
2. Sign the transaction. The signature is known before sending.
3. Insert the intent with `tx_sig` and status `signed`. Only then send.
4. On restart, for each intent not `confirmed`: query the signature status. Confirmed: mark it and write ledger rows. Not found and its blockhash has expired: mark `expired` and build a new intent attempt. Not found and blockhash still valid: wait.

Priority fee; confirm at `confirmed`; retries always go through this path.

8.7 Evidence

The canonical preimage, written once in `/docs/evidence.md` and used by both services:

```
atlas|v3|{gauge_id}|{value}|{event_time}|{run_id}|{storm_id}|{event_id}|{feed}|{slot}
```

- `value`: the latched triggering value, formatted to exactly 2 decimals (`"12.40"`).
- `event_time`: UTC, ISO 8601, seconds precision, trailing `Z` (`2026-10-11T03:15:00Z`).
- `feed`, `slot`: Switchboard feed pubkey and slot, or `-` if not used.
- Hash: SHA-256 of the UTF-8 preimage, lowercase hex. Memo text: `atlas:v3:<hash>`.
- The preimage is stored in `gauge_events`; the verify page recomputes the hash from it and compares with the Memo.

What it proves: the payout matches a recorded reading. It does not prove the reading is true; that is the oracle's job.

8.8 Bond token

Classic SPL mint; deposit is atomic (investor's aUSD to vault plus `mintTo` units at NAV, same signing flow as purchase). NAV from the ledger (7.4). Redemption is cut first; if asked, it is a multisig proposal after a notice period.

8.9 Reset (chain side)

Mint aUSD into vault and float up to the setup snapshot; update demo policy names and status to `ACTV`; release reservations; return balances to the engine for ledger entries and reconciliation.

8.10 Upgrades (after G2, 1 hour each, in order)

Upgrade	What to build	Drop it if
Solana Pay QR or Blink	Same buy endpoint. Blink needs `actions.json` at the domain root and CORS on every Action route; test on dial.to. Only possible with Flow A.	Flow B chosen, or not working on the phone in 1 hour
Squads spending limit	Daily aUSD limit for the bot key; bot pays through the spending-limit instruction instead of the float.	Not paying through it in 1 hour
Switchboard feed [F14]	One feed per demo gauge reading `GET /replay/feed/{gauge_id}`, which returns the *latched* value of that gauge's open event (or the last closed one), never the moving cursor. The bot cranks the feed after the event opens and pays only if the on-chain value equals the latched value and was updated at a slot after the event was created.	No on-chain value in 1 hour

9 Backend, frontend and hosting

9.1 Services

Service	Role	Owner
FastAPI	Read-only API over Tiger; replay start and reset orchestration; pricing; stress	A
Chain service (TS)	Purchase, bond deposit, Actions, `/chain/reset`, `/policies`	B
Payout bot (TS)	Interim and true-up payouts; intents; Memo	B
Streamer and poller (Python)	Live polling; replay; event and storm ticks	A
Web (Next.js or Vite)	Dashboard and phone view	B

All run on one Vultr instance under systemd or pm2, behind Caddy. Keep a full local copy runnable on a laptop.

9.2 Frontend

- Leaflet via `dynamic(() => import(...), { ssr: false })` in Next.js; wallet adapter in a `'use client'` provider; build against mocks first, real API from Hour 9.
- **Phone view is the money moment.** A page for one wallet in large type: policy status, balance, and a payout banner ("+$500 • PAID • Brays Bayou") that appears when the payout confirms, with the transaction link. It reads the wallet's aUSD balance directly from chain, so the number is real. Mirror it on the projector. Phantom is shown second, as proof the token sits in a real wallet.

Page	Shows
Map	Gauges coloured normal, near, triggered; replay clock (event time); storm status; Start and Reset only when an admin token is present
Gauge	Stage over event time with trigger line; events; rejected count; quote with band and freeze status; Buy
Pool	On-chain panel with seeded capital labelled and reconciliation badge; simulated panel with interim and final factors; NAV chart; payouts with Explorer links
Verify	Paste a payout signature; see preimage, recomputed hash, Memo hash, match
Phone view	One wallet: status, balance, payout banner

9.3 Hosting, security and abuse limits

- Caddy terminates HTTPS for the GoDaddy domain. If DNS has not propagated by Hour 4, use the `sslip.io` hostname for the Vultr IP and switch later.
- `POST /replay/*`, `/chain/reset` and any admin page require `X-Admin-Token`. Start and Reset buttons render only when the token is stored in the demo laptop's browser.
- `/chain/reset` listens on localhost only.
- Rate limits: buy and bond endpoints 5 requests per minute per IP; quote 60 per minute.
- Demo mode: buy and bond accept only allowlisted wallets; at most one active reservation per wallet; reservations expire after 2 minutes.
- Keys in environment files on the server, never in the repo or the browser.
- `actions.json` at the domain root if the Blink upgrade ships.

10 Demo script

The moment that wins: the bayou line turns red and the phone view shows the payout at the same instant.

Stage set-up: map on the laptop; phone view mirrored beside it; admin token loaded; fallback video open; **one backup policy already bought** on a second demo wallet for the chosen gauge [F21].

10.1 90-second version

0:00 to 0:15	"Maria rents in Meyerland. Federal flood claims take weeks. She needs money in the first 72 hours."
0:15 to 0:35	Tap a demo gauge button. Quote: probability, premium, band. Buy on the phone. If the purchase has not confirmed in 10 seconds, say "here's one we bought earlier" and switch to the backup wallet.
0:35 to 1:10	Start the replay. The map turns red; the phone view shows "+$500 • PAID".
1:10 to 1:30	Pool page: reserve used, bond absorbs the rest, reconciliation green. "Priced from decades of gauge data in Tiger Data, settled on Solana in seconds."

10.2 4-minute version

0:00 to 0:30	The gap, told through Maria. One sentence on Atlas.
0:30 to 1:15	Judge picks one of the 4 demo gauge buttons. Quote with band and the history behind it. Buy on the phone (backup ready). "Waiting period is off in demo mode."
1:15 to 2:30	Replay. Map turns red; payouts land in three wallets; phone view shows PAID. Storm closes; "everyone in the storm gets the same share." Pool page: reserve exhausted, NAV drops.
2:30 to 3:00	Verify one payout: preimage, hash, Memo match.
3:00 to 3:30	Numbers: worst-year capital need, simulated book interim and true-up factors, Tiger columnstore compression ratio.
3:30 to 4:00	Limitations in one breath (basis risk, licensing, cash-out), then the close.

Demo rules

Only offer the 4 `demo_ready` gauges. Press Reset before every judge group and check the reconciliation badge is green. Run one full replay 20 minutes before judging. Never debug in front of judges: switch to the fallback video within 15 seconds of anything breaking.

11 Judge Q&A preparation

Question	Answer
Your bot decides to pay. What does the chain add?	Purchase is one atomic transaction, so payment cannot be separated from the policy. Every payout carries a Memo hash of the exact reading it acted on. The pool's solvency and every payout are public. A Solana program enforcing the payout rule is the production step.
Is the payout fair in a big storm?	Every triggered policy gets the same interim share immediately, sized against all policies that could trigger, and a true-up when the storm closes. Order of crossing doesn't matter.
Which numbers are real?	On-chain panel: real devnet balances, with the starting capital labelled as seeded. Simulated panel: a 2,000-policy book, labelled on screen.
How does Maria cash out?	In this build she can't: off-ramps from a stablecoin to a bank or cash exist through third parties, but onboarding and cash-out are unsolved here. A production version would partner with an off-ramp or pay through a card linked to the wallet [F25].
What if the gauge triggers but my street stayed dry?	Basis risk, the core trade-off of parametric cover. We keep payouts small and fixed for immediate needs. Rainfall and multi-gauge triggers are next.
How did you size capital?	Each historical year is a scenario across all gauges at once, so correlation is built in. Capital covers the worst observed year above the mean. With a few decades of data that is a worst-year standard, not a modelled 1-in-200.

Question	Answer
Is your flood history comparable over time?	We checked each gauge's datum and channel changes and priced only from the comparable record; the decision per gauge is documented.
Is this legal?	Selling insurance needs a licence, and the bond token is likely a security. Atlas is the engine an insurer, city programme or nonprofit would run.
Can you fake the oracle?	In this build the bot reads Tiger and the replay is served by our server; we say so. With Switchboard, the bot checks the on-chain value matches the latched reading and was updated after the event. Production would use several sources.
Why not buy cover when a storm is coming?	7-day waiting period, and sales freeze while NWS has an active flood watch or warning at the gauge.
What stops double payment?	Every payout is signed and recorded as an intent before it is sent; a restarted bot checks the signature's status before doing anything.
Why does aUSD supply keep growing?	Reset mints test aUSD to restore the pool between demo runs. It is a test token; in production there is no reset.

12 Risk register, fallbacks and cut order

Risk	Odds	Switch trigger	Fallback
Phantom rejects Flow A	Medium	Hour-0 test fails	Flow B; drop Blink
Phantom hides aUSD or ignores metadata	Medium	Hour-0 phone check	Phone view is the money moment anyway
USGS legacy degraded	Medium	Any failed call, Hour 0 to 1	New API; else cache only, live poller dropped and said so
DNS not propagated	Medium	Not resolving at Hour 4	`sslip.io` hostname
Reset leaves pool wrong	Medium	Reconciliation fails	Fix before G3; G3 requires two clean runs
Running out of time	High	Missed gate	Cut order below
Live demo failure	Medium	Anything breaks	Fallback video, 15-second rule
Venue network or Vultr fails	Medium	Site unreachable	Local stack, hotspot
Switchboard will not work	Medium	No value after 1 hour	Memo-only evidence
Spending limit fights back	Medium	Not paying after 1 hour	Float funded by one proposal
Metadata update fails on rent	Low	Hour-0 test	Fixed width plus pre-funded space already in place
Purchase transaction too large	Low	Size error	Fewer metadata fields; pre-created mints

Risk	Odds	Switch trigger	Fallback
Columnstore API not on service	Low	Hour 0 version check	Old compression API
Tiger Cloud limits or outage	Low	Blocking error	Local Docker TimescaleDB

Cut order (first cut first): bond redemption; GEV fit; Switchboard; spending limit; Blink or QR (keep the Buy button); verify page; bond deposit flow (seed bond capital by script); true-up beat in the 4-minute script (keep the logic).

Never cut: time-shifted replay with reset and refill; ledger and reconciliation; admin token; the phone view payout; the map turning red; the pool page; the hosted URL; the backup policy; the fallback video.

13 Limitations of the product

13.1 Insurance design

- **Basis risk:** payouts follow a gauge, not your property.
- **Fixed benefit:** immediate needs only; does not replace flood insurance.
- **One payout per term:** back-to-back floods in one term pay once.
- **Partial interim payment [FC2]:** in a large storm the first payment can be a fraction of the benefit, with the rest paid when the storm closes, days later in a real event.
- **Adverse selection:** waiting periods and freezes reduce, but do not remove, buying ahead of known risk.

13.2 Data and oracle

- **Gauge failure** in the largest floods.
- **Provisional data** can be revised after a payout.
- **Single source**, itself mid-migration at USGS.
- **Record comparability:** datum and channel changes shorten the usable history for some gauges.
- **Replay trust:** the replay is served by the team's server and runs on shifted time.
- **Off-chain decision:** the bot decides to pay; the Memo makes that auditable, not enforced.

13.3 Pricing and capital

- Few extreme events per gauge; wide premium bands.
- Non-stationarity from development, drainage projects and climate change.
- Capital set to the worst observed year, not a modelled 1-in-200 tail.
- Catastrophe figures come from a simulated book; on-chain capital is seeded.
- Bond holders can lose most of their principal in one event.

13.4 Legal and regulatory

Licensed insurer or approved programme required; the bond token is likely a security; disclosure rules apply; payouts depend on a stablecoin and its issuer.

13.5 Users and adoption

- No cash-out in this build: the people most exposed often hold no wallet or stablecoin, and turning aUSD into rent money is unsolved.
- A lost wallet means a lost policy, since policies are non-transferable.
- People in a crisis may distrust an unfamiliar product.

13.6 Hackathon build

Devnet and test money; reset mints new test aUSD between runs; small gauge set; waiting period off in demo mode; custodial bot, mint and update keys; a hot-wallet float; no code audit.

14 Submission checklist

Submit by Hour 19.5; the deadline is 1:00 pm CDT, October 11 (Hour 24).

- ☐ Hosted URL on the GoDaddy domain (or documented `sslip.io` fallback), tested from a phone on mobile data.
- ☐ Admin controls hidden and protected on the public site.
- ☐ Public repository and README: problem, architecture, setup, run, reset and refill, evidence format, limitations.
- ☐ On-chain addresses: aUSD mint, example policy mint, multisig and vault, float wallet, bond mint, bot key, feeds if any, sample purchase, interim and true-up transactions.
- ☐ Demo video (2 to 3 minutes) with the phone view visible.
- ☐ Slides: Maria, demo, pricing, correlated capital, stress test, Tiger Data usage, Solana usage, limitations, next steps.
- ☐ All seven tracks selected (if allowed); each sponsor's usage described separately.
- ☐ `/docs/numbers.sql` reproduces every number on the slides; reconciliation passes.
- ☐ Reset pressed, one full run, badge green, 20 minutes before judging.

15 Unverified assumptions to check by Gate G1

Each item needs a yes or its fallback in `/docs/decisions.md`.

- ☐ Phantom on the demo phone accepts Flow A (server signs first). Otherwise Flow B **[F3]**.
- ☐ Phantom displays a Token-2022 policy's updated name, and shows aUSD with its name and symbol **[F4]**.

- ☐ USGS legacy endpoints answer for the chosen gauges; new API coverage; rate limits; history depth.
- ☐ Datum history and channel changes per gauge **[F11]**.
- ☐ NWS flood-stage access; NWS alerts by point.
- ☐ Exact Harvey window, reading counts and the 4 gauges that cross trigger, from stored data **[F20][F22]**.
- ☐ Token-2022 extension combination, name and field updates at fixed width, initialization order, extra rent **[F13]**.
- ☐ Purchase transaction under 1,232 bytes.
- ☐ Squads v4 multisig, vault token account and time lock on devnet with `@sqds/multisig` on `@solana/web3.js` 1.x.
- ☐ TimescaleDB version on Tiger Cloud supports the columnstore API **[F19]**; free-tier limits; whether the track requires Tiger Cloud.
- ☐ Vultr and GoDaddy Registry track rules; Devpost multi-track entry **[F24]**.
- ☐ Domain resolving, or `sslip.io` fallback working with Caddy **[F18]**.

Sources

Carried over from v2 (not re-checked for v3):

- USGS Water Data APIs (legacy WaterServices retirement and possible degradation): `https://www.usgs.gov/tools/usgs-water-data-apis`
- USGS API decommission notice: `https://waterdata.usgs.gov/blog/api-decom-fall-2025/`
- Circle testnet faucet (devnet USDC limits): `https://developers.circle.com/w3s/docs/developer-console-faucet`
- CodeRED: Orion on Devpost (tracks, rules, deadline): `https://codered-orion-ccs.devpost.com/`